МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Интеллектуальные методы анализа данных»

Тема: Метод нечеткой кластеризации с-средних

Студент гр. 3384 _____ Горский К.Д.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Реализация метода нечеткой кластеризации с-средних с использованием природных временных рядов.

Задание.

Вариант 5. Станция Туле (THUL)

1. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам в Приложении 1 табл. п.1 и п.2. Использовать данные с одноминутным временным разрешением.

2. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).

3. Сформировать матрицу входных данных data из преобразованных временных рядов в фиксированное число признаков (например, среднее значение, стандартное отклонение, амплитуда и др.).

4. Реализовать метод нечеткой кластеризации с-средних. В MatLab воспользоваться функцией: *[centers, U] = fcm(data, Nc, options)*, где data — входные данные (строки data соответствуют наблюдениям), Nc — количество кластеров, options — параметры метода.

5. Выполнить проверку работоспособности построенного метода кластеризации.

6. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Работа выполнялась на языке Python: numpy и pandas для обработки данных, matplotlib для визуализации, нечеткая кластеризация — функцией cmeans пакета scikit-fuzzy. Исходный код см. в приложении А.

Данные станции Туле (THUL) скачаны с одноминутным разрешением. По таблице из условия лабораторной работы отобраны 45 спокойных суток (44 с удалением повтора) и 45 суток с магнитными бурями. Каждые сутки представляют собой временной ряд из 1440 отсчётов.

Пропуски в рядах заполняются медианным значением соответствующего ряда: ряд переиндексируетсяна полную сетку минут, недостающие отсчёты заменяются на медиану.

Каждый суточный ряд преобразовывается в вектор из 10 признаков: среднее значение, стандартное отклонение, амплитуда (max - min), медиана, минимум, максимум, 25-й и 75-й процентили, 1-й и 99-й процентили.

Так сформировывается матрица входных данных X. Перед кластеризацией признаки стандартизованы (нулевое среднее, единичная дисперсия), чтобы привести все признаки к одному масштабу.

Кластеризация выполнена нечетким методом с-средних при Nc = 2 кластерах, параметре размытости m = 2, точности 10^{-4}. Номер кластера для каждого наблюдения получен как argmax по строке результирующей матрицы принадлежностей U, (т. е. для каждого вектора выбирается кластер с наибольшим весом).

Нумерация кластеров произвольна, поэтому при сравнении с истинными метками кластеры сопоставлялись так, чтобы accuracy была максимальной. Если accuracy < 0.5, то кластеры перенумеруются.

Качество кластеризации оценивалось по матрице ошибок, accuracy, recall и precision для класса «магнитная буря», а также по индексу размытости FPC (fuzzy partition coefficient). FPC — это специальная метрика, вычисляющаяся как квадрат нормы Фробениуса результирующей матрицы принадлежностей. Она показывает «уверенность» модели в присвоении классов точкам (1 — уверенность большая, 1/c — уверенности нет, каждый класс для каждой точки равновероятен).

Результаты приведены в таблицах 1–2 и на рис. 1.

Таблица 1 — Метрики качества кластеризации

Метрика	Значение
accuracy	0.539
recall (буря)	0.182
precision (буря)	0.615
FPC	0.869

Таблица 2 — Матрица ошибок (строки — истинный класс, столбцы — предсказанный)

Предсказанный \ Истинный	буря	спокойно
буря	8	36
спокойно	5	40

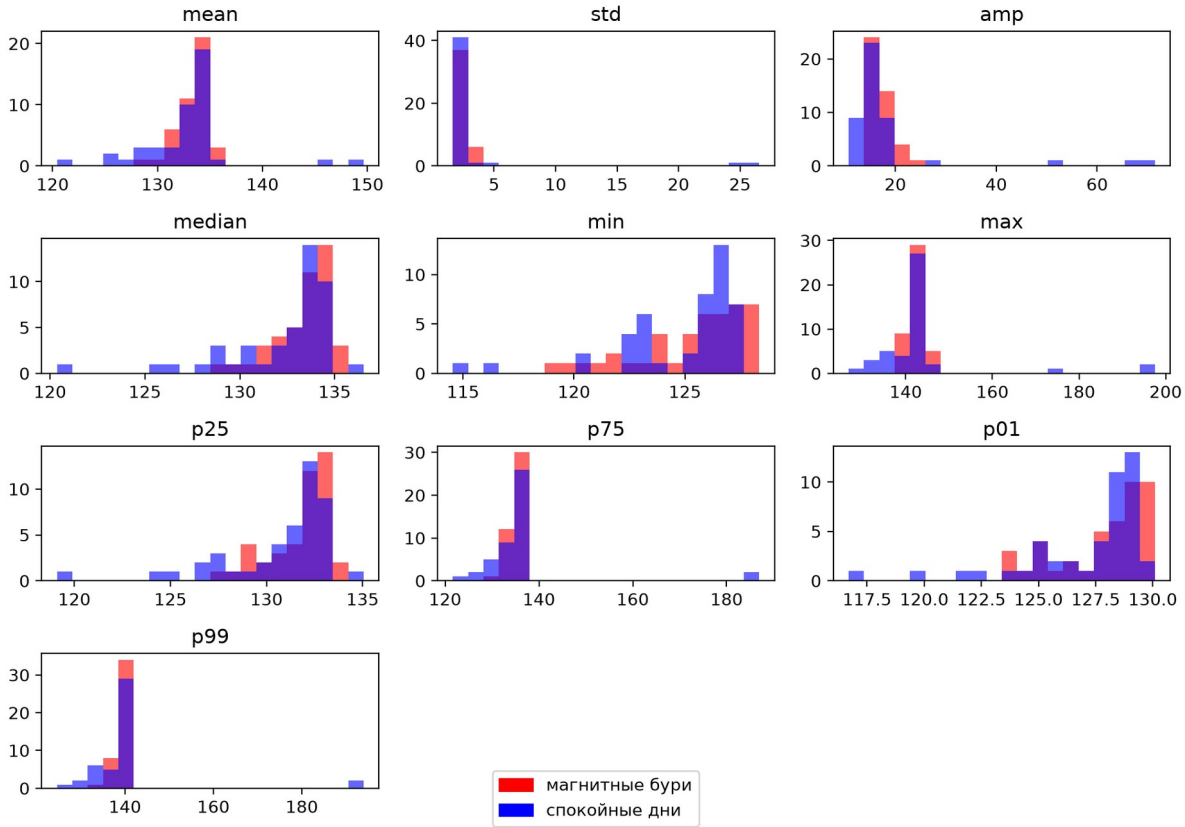


Рисунок 1 — Гистограммы распределений признаков (истинные)

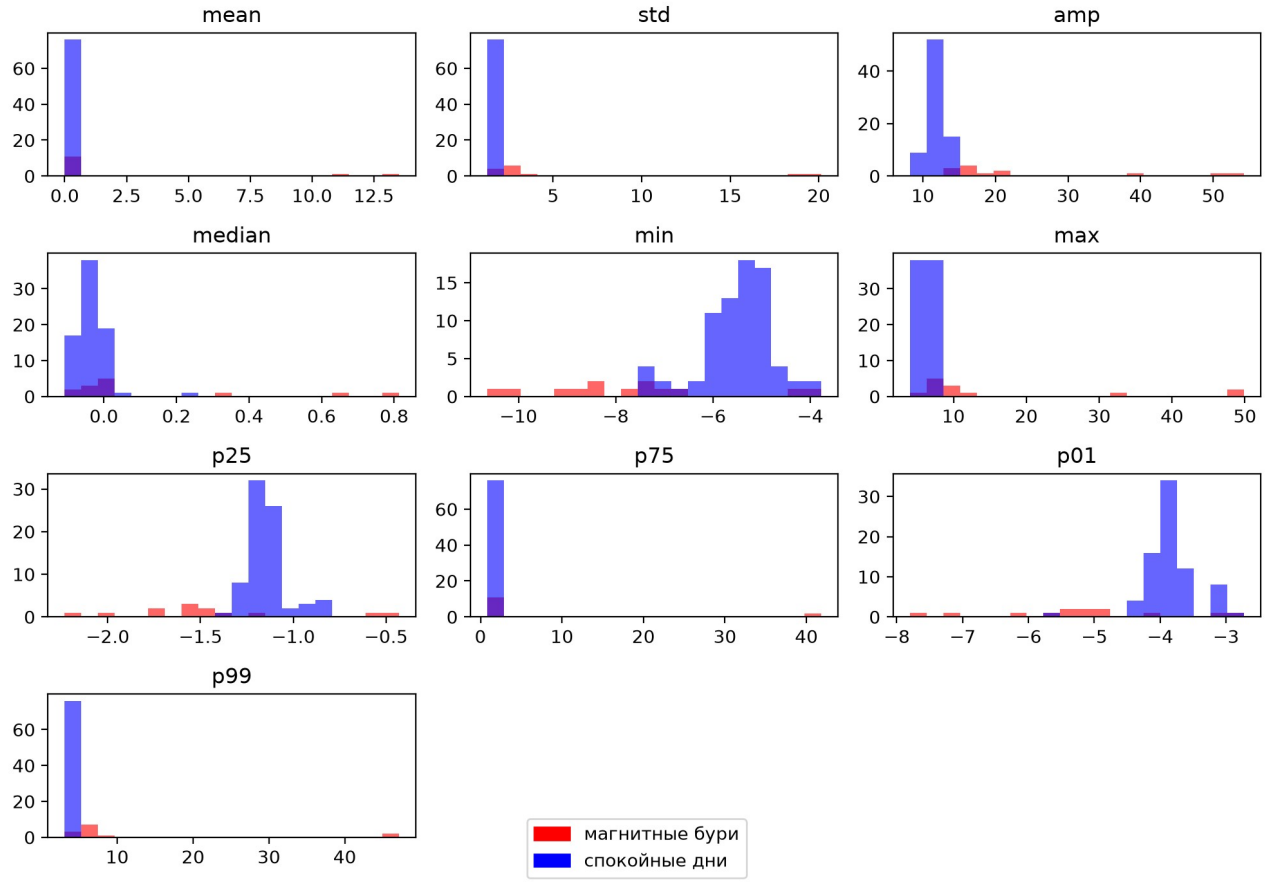


Рисунок 2 — Гистограммы распределений признаков (предсказанные)

На рис. 1 и 2 mean, std, amp, median, min, max, p25, p75, p01, p99 — это среднее, отклонение, амплитуда, медиана, минимум, максимум, процентили соответственно.

Accuracy кластеризации составила 0.539 — лишь немного выше случайного угадывания. Из матрицы ошибок видно, что с-средних отнес к кластеру «буря» только 13 суток (8 бурь и 5 спокойных), а остальные 76 суток объединил в один большой кластер. Recall класса «буря» равен 0.182: распознано лишь 8 из 44 возмущённых суток, тогда как 40 из 45 спокойных попали в свой кластер. Таким образом, алгоритм выделяет небольшой хвост в отдельный кластер, но не воспроизводит истинные кластеры.

FPC = 0.869 указывает на достаточно чёткое разбиение (при полностью размытом разбиении FPC = 0.5), однако это разбиение почти однокластерное:

основная масса суток сливается в один кластер, а второй кластер собирает лишь крайние по форме профиля сутки. На гистограммах (рис. 1) распределения признаков классов перекрываются по всем десяти признакам, визуально кластеры выделить невозможно. Алгоритм с-средних с задачей тоже не справился.

Выводы.

Реализован метод нечеткой кластеризации с-средних с использованием природных временных рядов.

Accuracy кластеризации составила 0.539, recall класса «буря» — 0.182, precision — 0.615, индекс размытости FPC — 0.869. Кластер, соответствующий бурям, собрал лишь 13 суток из 89, остальные 76 суток объединены в один большой кластер; при этом 40 из 45 спокойных суток распознаны верно, а из 44 возмущенных — только 8. Кластеры, найденные методом с-средних, не соответствуют размеченным данным. Рспределения выбранных признаков классов сильно перекрываются, поэтому алгоритм выделяет небольшой кластер аномальных суток на фоне неразделимой основной массы. Выбранные признаки недостаточно информативны для разделения классов.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import numpy as np
import pandas as pd
import skfuzzy
from pathlib import Path

SEED = 123
FEATURE_NAMES = [
    "mean", "std", "amp", "median",
    "min", "max",
    "p25", "p75", "p01", "p99",
]
M=2

def load_day(path: Path):
    df = pd.read_csv(path)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp")
    full = pd.date_range(df.index.min().normalize(), periods=1440, freq="1min")
    df = df.reindex(full)
    median = float(df["RCORR_E"].median())
    df["RCORR_E"] = df["RCORR_E"].fillna(median)
    return df["RCORR_E"].to_numpy(dtype=float)

def features(s):
    m = float(s.mean())
    sd = float(s.std())
    med = float(np.median(s))
    p25, p75 = np.percentile(s, [25, 75])
    p01, p99 = np.percentile(s, [1, 99])
    return np.array([
        m, sd, s.max() - s.min(), med,
        s.min(), s.max(),
        p25, p75, p01, p99,
    ])

def load_all():
    X, y = [], []
    for cls in (0, 1):
        for path in sorted((Path("data") / f"class{cls}").glob("*.csv")):
            series = load_day(path)
            X.append(features(series))
            y.append(cls)
    return np.array(X), np.array(y)

def match_to_pred_labels(y_true, y_pred):
    acc1 = float(np.mean(y_pred == y_true))
    acc2 = float(np.mean(y_pred == 1 - y_true))
    if acc2 > acc1:
        return acc2, 1 - y_pred
    return acc1, y_pred

def confusion(y_true, y_pred):
    return np.array([[np.sum((y_true == 0) & (y_pred == 0)),
                      np.sum((y_true == 0) & (y_pred == 1))],
                     [np.sum((y_true == 1) & (y_pred == 0)),
                      np.sum((y_true == 1) & (y_pred == 1))]])
```

```
def draw_gist(ys, filename):
    cols = {0: "#FF0000", 1: "#0000FF"}
    fig, axes = plt.subplots(4, 3, figsize=(10, 7))
    for i, ax in enumerate(axes.flat):
        if i >= len(FEATURE_NAMES):
            ax.axis("off")
            continue

        f0, f1 = X[ys == 0, i], X[ys == 1, i]
        bins = np.histogram_bin_edges(np.concatenate([f0, f1]), bins=20)
        ax.hist(f0, bins=bins, alpha=0.6, color=cols[0])
        ax.hist(f1, bins=bins, alpha=0.6, color=cols[1])
        ax.set_title(FEATURE_NAMES[i])

    fig.legend(handles=[Patch(color=cols[0], label="магнитные бури"),
                        Patch(color=cols[1], label="спокойные дни")],
            loc="lower center")
    fig.tight_layout()
    fig.savefig(filename, dpi=200)
    plt.close(fig)

X, y = load_all()

std = X.std(axis=0)
std[std == 0] = 1.0
Xn = (X - X.mean(axis=0)) / std

cntr, U, _, _, _, p, fpc = skfuzzy.cmeans(Xn.T, c=2, m=M, error=1e-4,
maxiter=100, seed=SEED)
print(f"converged in {p} iters")

y_pred = U.argmax(axis=0)
acc, y_pred = match_to_pred_labels(y, y_pred)
cm = confusion(y, y_pred)

print(f"accuracy: {acc:.3f}")
print("confusion matrix:")
print(cm)

tp, fn, fp, tn = cm[0, 0], cm[0, 1], cm[1, 0], cm[1, 1]
recall = tp / (tp + fn)
precision = tp / (tp + fp)
print(f"recall (бури): {recall:.3f}")
print(f"precision (бури): {precision:.3f}")
print(f"FPC: {fpc:.3f}")

draw_gist(y, "img/feat_hists.png")
draw_gist(y_pred, "img/feat_hists_pred.png")
```